

Classification with Naive Bayes

این پروژه یک مدل یادگیری ماشین برای پیش‌بینی ابتلا به دیابت بر اساس مجموعه داده‌های پزشکی مربوط به زنان بومی پاما (Pima Indians) است. هدف اصلی این پروژه این است که با استفاده از ویژگی‌هایی مانند سطح گلوکز خون، فشار خون، سن، شاخص توده بدنی (BMI)، تعداد بارداری‌ها و...، پیش‌بینی کنیم که آیا یک فرد مبتلا به دیابت است یا خیر.

در این پروژه مراحل زیر انجام می‌شوند:

بارگذاری و بررسی داده‌ها از یک فایل CSV خوانده شده و ویژگی‌ها مورد بررسی قرار می‌گیرند.

آماده‌سازی داده‌ها ستون خروجی (Outcome) به‌عنوان هدف مدل انتخاب شده و داده‌ها به دو بخش آموزشی و آزمایشی تقسیم می‌شوند.

آموزش مدل از الگوریتم Naive Bayes برای آموزش مدل روی داده‌های آموزشی استفاده می‌شود.

پیش‌بینی و ارزیابی عملکرد مدل با استفاده از ماتریس اشتقاق (Confusion Matrix)، دقت (Accuracy)، فراخوانی (Recall)، دقت پیش‌بینی (Precision) و امتیاز F1 بررسی می‌شود.

[38]
✓ 0s

```
df = pd.read_csv("cardata.csv")  
df.head()
```

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0

Next steps: [Generate code with df](#) [New interactive sheet](#)

ستون	توضیح
Pregnancies	تعداد بارداری‌ها
Glucose	سطح گلوکز در خون
BloodPressure	فشار خون
SkinThickness	ضخامت پوست
Insulin	سطح انسولین
BMI	شاخص توده بدنی (وزن تقسیم بر مجذور قد)
DiabetesPedigreeFunction	سلیقه خانوادگی دیابت
Outcome	نتیجه نهایی (۰ = بدون دیابت، ۱ = مبتلا به دیابت)

[39]
✓ Os

df.shape

▼

(301, 9)

[40]
✓ Os

df.describe()

▼

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
count	301.000000	301.000000	301.000000	301.000000	301.000000
mean	2013.627907	4.661296	7.628472	36947.205980	0.043189
std	2.891554	5.082812	8.644115	38886.883882	0.247915
min	2003.000000	0.100000	0.320000	500.000000	0.000000
25%	2012.000000	0.900000	1.200000	15000.000000	0.000000
50%	2014.000000	3.600000	6.400000	32000.000000	0.000000
75%	2016.000000	6.000000	9.900000	48767.000000	0.000000
max	2018.000000	35.000000	92.600000	500000.000000	3.000000

[41]
✓ Os

X = df.drop("Owner", axis=1)
y = df["Owner"]

Convert categorical columns to numerical using one-hot encoding
X = pd.get_dummies(X, columns=['Car_Name', 'Fuel_Type', 'Seller_Type', 'Transmission'], drop_first=True)

در اینجا ما ستون ویژگی‌ها (X) را از برجسب هدف (y) جدا می‌کنیم.

چون هدف ما پیش‌بینی بیماری دیابت است، ستون "Outcome" را جدا کرده و از بقیه ویژگی‌ها برای آموزش مدل استفاده می‌کنیم.

```
[42] ✓ Os from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42 # Removed stratify=y
)

print("X_train Shape: ",X_train.shape)
print("X_test Shape: ",X_test.shape)
print("y_train Shape: ",y_train.shape)
print("y_test Shape: ",y_test.shape)

X_train Shape: (225, 105)
X_test Shape: (76, 105)
y_train Shape: (225,)
y_test Shape: (76,)
```

ما مجموعه داده را به دو بخش تقسیم می‌کنیم:

آموزشی (Train): برای آموزش مدل استفاده می‌شود.

آزمون (Test): برای ارزیابی مدل استفاده می‌شود.

در اینجا، 600 نمونه برای آموزش و 168 نمونه برای آزمون انتخاب شده‌اند.

◆ Gemini

```
[43] ✓ Os from sklearn.naive_bayes import GaussianNB
nb = GaussianNB()
```

```
[44] ✓ Os nb.fit(X_train,y_train)
```

▾ GaussianNB ⓘ ⓘ
GaussianNB()

در این مرحله ما از مدل Naive Bayes (گوسی) برای طبقه‌بندی استفاده می‌کنیم. این مدل فرض می‌کند که ویژگی‌ها مستقل از یکدیگر هستند

و به صورت توزیع نرمال هستند.

```
[45] ✓ Os nb.predict(X_test)[:10]

array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

با استفاده از مدل آموزش‌دیده، ما خروجی 10 نمونه‌ی اول را پیش‌بینی می‌کنیم. خروجی‌ها شامل اعداد ۰ یا ۱ هستند (تشخیص دیابت یا عدم آن).

```
[46] from sklearn.metrics import confusion_matrix, accuracy_score, roc_auc_score, roc_curve, classification_report, mean_squared_error, f1_score, recall_score, precision_score
✓ 0s y_pred = nb.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
cm
array([[74,  0],
       [ 2,  0]])
```

ماتریس درهم‌ریختگی عملکرد مدل را نشان می‌دهد:

پیش‌بینی: 1 پیش‌بینی: 0

واقعاً 0 TP = 92 FP = 16

واقعاً 1 FN = 24 TN = 36

```
[47] accuracy_score(y_test, y_pred)
✓ 0s
```

0.9736842105263158

```
[48] recall_score(y_test, y_pred)
✓ 0s
```

0.0

```
[49] precision_score(y_test, y_pred)
✓ 0s
```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 due to no predicted samples. Use _warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))

0.0

```
[50] f1_score(y_test, y_pred)
✓ 0s
```

0.0

[51]
✓ 0s

```
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.97	1.00	0.99	74
1	0.00	0.00	0.00	2
accuracy			0.97	76
macro avg	0.49	0.50	0.49	76
weighted avg	0.95	0.97	0.96	76

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted sam
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted sam
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted sam
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

معیار	توضیح
Precision	از میان همه پیش‌بینی‌های مثبت، چند درصد درست بودند؟
Recall (Sensitivity)	از میان همه نمونه‌های واقعاً مثبت، چند درصد را درست تشخیص داد؟
F1-score	مقیاد برای داده‌های نامتوازن، میانگین هارمونیک Precision و Recall